

Test Preparation Materials

TABLE OF CONTENTS	
No	Topic
1	<u>HTML / CSS / Bootstrap</u>
2	<u>JavaScript</u>
3	<u>NodeJS</u>
4	<u>VueJS</u>
5	<u>Git</u>
6	<u>Misc</u>
7	<u>Flutter</u> (Bonus)

JavaScript

Take [this](#) JS crash course to revise.

Your interview questions will be from the following links

[\[0\]](#) [\[1\]](#) [\[2\]](#) [\[3\]](#) [\[4\]](#) [\[5\]](#) [\[6\]](#)

1. What is the Event Loop [\[1\]](#)
2. How would you make a copy of an array [\[1\]](#)
3. JSON, XML & AJAX [\[1\]](#) [\[2\]](#) [\[3\]](#) [\[4\]](#)
4. DOM [\[1\]](#) [\[2\]](#)
5. Push, Pop, Shift, Unshift
6. Closure in JS [\[1\]](#) [\[2\]](#)
7. Array manipulation [\[1\]](#) [\[2\]](#) [\[3\]](#)
8. Higher order functions on arrays (map for JS) [\[1\]](#) [\[2\]](#)
9. Higher order functions - Filter & reduce method [\[1\]](#) [\[2\]](#) [\[3\]](#)
10. Functional programming [\[1\]](#)
11. String Manipulation
12. Parsing strings [\[1\]](#) [\[2\]](#) [\[3\]](#)
13. Loading JSON data from a URL (Async Callbacks) [\[1\]](#)
14. Axios [\[1\]](#) [\[2\]](#)
15. Sockets [\[1\]](#)
16. Can I use Javascript to detect if there is an internet connection?
17. When a user is logged in, How would you save their authentication information?
18. What can sockets be used for?
19. How to get the value of a certain key from a Javascript object? [\[1\]](#)
20. Decoding encoded bit data from a variable
21. How would you remove duplicate values from an array [\[1\]](#) [\[2\]](#)
22. What is the **'this'** keyword in JavaScript?
23. What are exceptions? [\[1\]](#)
24. How are exceptions handled in JavaScript? [\[1\]](#) [\[2\]](#) [\[3\]](#) [\[4\]](#)
25. Which is faster if/else statement or switch case (and why ?)
26. Must know how to debug JavaScript [\[1\]](#)
27. LinkedList [\[1\]](#)
28. Difference between Callbacks vs Promises vs Async/await? [\[1\]](#) [\[2\]](#)
29. What are the drawbacks of Callbacks? [\[1\]](#) [\[2\]](#)
30. Promises
31. What is a drawback of Promises
32. Cache
33. AMP [\[1\]](#)
34. Offline Canvas in Chrome [\[1\]](#)

- 35. Service workers & Caching [\[1\]](#) [\[2\]](#) [\[3\]](#)
- 36. The Service Worker Lifecycle [\[1\]](#)
- 37. Debugging Service workers using DevTools [\[1\]](#)
- 38. How can we compress data in Javascript [\[1\]](#)
- 39. Create an interactive JavaScript 3d Model [\[1\]](#)
- 40. Build a 360-view image slider with JavaScript [\[1\]](#)
- 41. What is the difference between a linter and a formatter?
- 42. What formatting tools are available [\[1\]](#)

NodeJS

Please look at this [\[Link\]](#)

- 43. What is Node [\[1\]](#)
 - 44. What is NPM ? [\[1\]](#) [\[2\]](#) [\[3\]](#)
 - 45. Callbacks & Promises in NodeJS [\[1\]](#) [\[2\]](#) [\[3\]](#)
 - 46. Faster Async functions and promises [\[1\]](#)
 - 47. Sequelize.JS (ORM) [\[1\]](#) [\[2\]](#) [\[3\]](#) [\[4\]](#) [\[5\]](#) [\[6\]](#)
 - 48. TypeORM [\[1\]](#)
 - 49. What are migrations [\[1\]](#)
 - 50. What are seeds [\[1\]](#) [\[2\]](#)
 - 51. Building a RESTFUL API [\[1\]](#) [\[2\]](#)
 - 52. Auto Generating API docs with Node & Swagger [\[1\]](#) [\[2\]](#) [\[3\]](#) [\[4\]](#)
 - 53. Garbage collection [\[1\]](#)
 - 54. How can we use migrations [\[1\]](#)
 - 55. Express.js Framework For Node.js [\[1\]](#) [\[2\]](#) [\[3\]](#) [\[4\]](#)
 - 56. Fastify [\[1\]](#)
 - 57. What is a framework [\[1\]](#)
 - 58. Fastify vs Express [\[1\]](#)
 - 59. What is NestJS [\[1\]](#)
 - 60. What are Race Conditions and how to handle it [\[1\]](#)
 - 61. Push Notifications using NodeJS + web-push [\[1\]](#)
 - 62. Unit testing (Jest, Mocha, Jasmine)
 - 63. Bundlers, Task runners, Build / Dev tooling [\[Grunt, Gulp, NPM, Webpack\]](#)
 - 64. Google Sheets for Node JS developers [\[1\]](#)
- [\[50 Interview questions & answers for NodeJS\]](#)

VueJS

Don't know VueJs? Click [here](#)

[\[Program with Erik\]](#)

[\[Why do FrontEnd with VueJS\]](#)

[\[Vue over react\]](#)

[\[Vue.js 2 in 60 minutes\]](#)

[\[Learn Vue.js in three hours\]](#)

[\[10 Tips to become a better VueJS developer\]](#)

[\[Intro to Vue - Build a to-do app in 20 minutes\]](#)

- 65. Why Choose VueJS or React? [\[1\]](#) [\[2\]](#) [\[3\]](#)
- 66. What are VueJS components
- 67. How are VueJS components created

68. VueJS Components LifeCycle
69. Routing in VueJS
70. What is the difference between **v-if** and **v-show**
71. What is the **updated** function (what are its pros and cons)
72. Filters in VueJS
73. Fragments in Vue [\[1\]](#)
74. Plugins
75. Mixins [\[1\]](#)
76. What is single Page Applications in Vue
77. Declarative rendering
78. Class and style binding
79. Event handling
80. How to pass data between components
81. Props
82. Type of loops in VueJS
83. How many ways can props be declared
84. What are the pros and cons of using those different ways
85. Attributes
86. How are attributes used
87. Vue Apollo (GraphQL with Vue) [\[1\]](#)
88. Vuex [\[1\]](#) [\[2\]](#)
89. Vue-DevTools [\[1\]](#)
90. Debugging VueJS [\[1\]](#)
91. VueJS Dropdown with Keyboard Accessibility [\[1\]](#)
92. VueJS search with vue-fuse [\[1\]](#)
93. How are HTTP requests sent via Vue
94. Vue Testing with Jest [\[1\]](#)
95. Unit testing [\[1\]](#)
96. State management in Vue
97. Lifecycle Hooks
98. Static site generation for Vue (Gridsome) [\[1\]](#) [\[2\]](#) [\[3\]](#)
99. Using Gridsome with Wordpress [\[1\]](#)
100. Server-side rendering (SSR) with VueJs
101. Faster Web applications with Vue 3 [\[1\]](#) [\[2\]](#)
102. Vue 3 Composition API [\[1\]](#)
103. Vite

HTML5 /CSS /BootStrap /Jquery

[Beginners HTML5]

104. Demonstrate excellent proficiency in HTML5, CSS3 [\[1\]](#) [\[2\]](#) [\[3\]](#)
105. What is HTML canvas [\[1\]](#) [\[2\]](#) [\[3\]](#)
106. Speed up HTML with Emmet [\[1\]](#) [\[2\]](#)
107. Bootstrap 4 [\[1\]](#)
108. What is an offset in Bootstrap [\[1\]](#) [\[2\]](#) [\[3\]](#) [\[4\]](#)
109. Media queries in Bootstrap [\[1\]](#) [\[2\]](#) [\[3\]](#) [\[4\]](#)
110. What is z-index (CSS) [\[1\]](#) [\[2\]](#)
111. Standard media queries [\[1\]](#)
112. CSS transitions [\[1\]](#)
113. Drawing + Animations on Canvas
114. What is Webkit in CSS [\[1\]](#) [\[2\]](#) [\[3\]](#)
115. CSS Grid [\[1\]](#) [\[2\]](#) [\[3\]](#) [\[4\]](#) [\[5\]](#)
116. Flexbox (CSS) [\[1\]](#) [\[2\]](#) [\[3\]](#) [\[4\]](#)
117. Responsive design (viewport & media queries) [\[1\]](#) [\[2\]](#) [\[3\]](#)
118. CSS3 Viewport [\[1\]](#) [\[2\]](#) [\[3\]](#)
119. I have 10 rows in a table, how can I change the color of the 4th row without using a class
120. JQuery [\[1\]](#) [\[2\]](#) [\[3\]](#)
121. Why do we get \$undefined errors [\[1\]](#)
122. How can you play a video using HTML5 [\[1\]](#)
123. What are iframes [\[1\]](#) [\[2\]](#) [\[3\]](#) [\[4\]](#)
124. How can you add 360 degree images on HTML page [\[1\]](#)
125. Tricky CSS properties (Box Model,Font-relative, viewport etc) [\[1\]](#)
126. Must understand and know CSS tricks [\[1\]](#) [\[2\]](#)
127. Knowledge of CSS Preprocessors (Ideally Sass) [\[1\]](#) [\[2\]](#)
128. What are CSS image sprites [\[1\]](#) [\[2\]](#) [\[3\]](#) [\[4\]](#) [\[5\]](#) [\[6\]](#)
129. 3d models in HTML documents [\[1\]](#) [\[2\]](#)
130. Can I build a 360 view of an object using HTML
131. Basic understanding of Google Developer Tools [\[1\]](#)
132. How to improve load performance using Chrome DevTools [\[1\]](#)
133. Supercharging Page load [\[2\]](#)
134. Designing for speed & hacking user perception [\[1\]](#)
135. Advanced Chrome page load speed [\[1\]](#)
136. Charting Libraries D3, HighChart [\[1\]](#)

Git

- 137. Git & Github Crash Course [\[1\]](#)
- 138. Github [\[1\]](#)
- 139. What is GitFlow [\[1\]](#)
- 140. Resolving conflicts during Git merge [\[1\]](#)
- 141. Git Graph [\[1\]](#)
- 142. Git rebase [\[1\]](#)[\[2\]](#)
- 143. Git Submodule
- 144. Git squash

Database

- 145. MySql
- 146. PostGres
- 147. mongoDB
- 148. Sqlite
- 149. Indexing

Misc Section

- 150. Linux basic command line [\[1\]](#) [\[2\]](#) [\[3\]](#)
- 151. Bash [\[1\]](#)
- 152. SSH [\[1\]](#) [\[2\]](#)
- 153. VIM [\[1\]](#)
- 154. HTTP Crash course [\[1\]](#)
- 155. Google Chrome Dev tools crash course [\[1\]](#)
- 156. **What is a REST API?** [\[1\]](#)
- 157. **REST vs GraphQL** [\[1\]](#)
- 158. REST vs RPC vs GraphQL [\[1\]](#)
- 159. **GraphQL** [\[1\]](#)
- 160. **MQTT** Protocol [\[0\]](#) [\[1\]](#) [\[2\]](#) [\[3\]](#) [\[4\]](#) [\[5\]](#)
- 161. **MQTT** Mosquitto broker [\[1\]](#)
- 162. **MQTT** Publishers / Subscribers [\[1\]](#)
- 163. tRPC [\[1\]](#)
- 164. Sending Curl
- 165. Nginx [\[1\]](#)
- 166. WebSockets [\[1\]](#)
- 167. What are Progressive Web Apps (PWAs)?
- 168. What is Webpack? [\[1\]](#) [\[2\]](#)
- 169. What's the difference between Webpack / Gulp / Grunt ?
- 170. What is tree shaking?
- 171. Must understand MVPs [\[1\]](#)
- 172. What is docker [\[1\]](#)
- 173. Must understand the difference between prototyping and building for production
- 174. Basic understanding of DevOps
- 175. Basic understanding of Kubernetes / Spinnaker / Pipelines
- 176. Will prefer someone with an understanding of software testing
- 177. Working knowledge of Git (Github / bitbucket etc)
- 178. Working knowledge of using Google Docs
- 179. Working knowledge of using Slack
- 180. Be able to create short videos/screengrabs using Nimbus Screen recorder or any alternative software. Must be able to quickly (and asynchronously communicate with the team via Slack / Whatsapp / short videos / well-written code & Good documentation.
- 181. Must be able to demonstrate ability to work as a team player. This means you should have good coding practices and should document/comment on your code properly. Being a good team member also means the ability to communicate effectively.

Here are some free resources that can help you. For fresh candidates, it's okay if you don't know everything. For experienced candidates, we expect more.

[\[Net Ninja\]](#)

[\[Dylan Israel\]](#)

[\[Coding Garden with CJ\]](#)

[\[Coding train\]](#)

[\[Functional JS with Fun Fun Function\]](#)

Flutter (Optional)

Here is a free Flutter [playlist](#) that is very good to learn and revise from. If we hire you we have multiple paid courses that you'll be able to access.

[\[How Flutter works in 8 min\]](#)

[\[Flutter - Widget Tree & Element Tree in 8 min\]](#)

[\[Flutter Cookbook\]](#)

182. Why Flutter, tell 5 pros and cons of flutter?
183. Why does Flutter use Dart ? [\[1\]](#)
184. Basic Dart programming and tools. [\[1\]](#) [\[2\]](#) [\[3\]](#)
185. Abstraction, Inheritance, polymorphism and encapsulation concepts in dart.
186. What is Pubspec.yaml file
187. Difference between Stateful and Stateless Widgets?
188. Asynchronous programming in flutter?and why do we need that?
189. What is Await in flutter?
190. Difference between then() and whenComplete() function?
191. Use of Native widgets for IOS and Android.
192. Should be able where and how to add Dart packages in flutter app.
193. How to pass data/variables across widgets in flutter app.
194. Use of Futures,Async, Await and timeout in flutter. [\[1\]](#)
195. Difference Between async and async* ?
196. State Management in flutter via Getx, Provider and other methods.
197. What is Isolates ?
198. JSON Manipulation (parsing, encoding) in flutter.
199. Knowledge about Web APIs (REST, SOAP) and consuming API calls in flutter.
200. Knowledge about GET,POST, PUT,CURL requests.
201. Should have knowledge about SQLite (or SQLite plugin in flutter) and all the basic database concepts and operations(CRUD).
202. Material App and how to use it in flutter.
203. Understanding of Android Studio project build, xml and other files, where to add permissions and dependencies for android and IOS project.
204. Callbacks in flutter.
205. Deep Understanding about Lists and Maps in dart.
206. Basic knowledge about firebase or any other commonly used cloud service to integrate with flutter app for the data exchange between app and cloud, or for push notifications etc.
207. Secrets in Flutter [When to Use Keys - Flutter Widgets 101 Ep. 4 How To Store API Keys For Staging & Production Environments](#)
208. Navigation to different screens in flutter (routes concept). [\[1\]](#) [\[2\]](#)
209. Local Push Notifications [\[1\]](#) [\[2\]](#)
210. How can you communicate Between Dart and Native code in flutter?

- 211. What is Pump and PumpSettle in flutter Testing?
- 212. Flutter Dev tools [\[1\]](#)
- 213. What is flutter for Web [\[1\]](#)

MUST PREPARE FOR THE INTERVIEW

Must have clear concepts!

1. GIT
2. Unit Test and Integration testing
3. Docker
4. API Integration
5. .env variables
6. ORMS
 - a. Sequelize
 - b. Prisma
 - c. Mongoose
7. **ADDITION POINTS**
 - a. MQTT
 - b. Influx
 - c. GitLab - CI/CD
 - d. Github Actions

DO NOT LEAVE THIS SECTION

e.

Cowlar Software (Web Dev track) training [Document](#)